

ECS401: Unit 5 Lab Progress Test

FEEDBACK: Answers and Explanations

If you got any wrong you urgently need to do something to improve your understanding before it is too late.

Write a fragment of code to do each of the following:

1. Declare an array called `teams` that holds 10 Strings.

CORRECT ANSWER:

```
String[] teams = new String[10];
```

EXPLANATION:

An array of a given type is declared by giving that type and then square brackets to indicate the array.

The first part of this is therefore just like a normal declaration of the form

type name-of-variable

With an array we then have to create the space (on the heap) big enough to hold it. That is done using keyword `new`. We use assignment to store a reference to this new object in the array variable (on the stack) we just created. Here we want to create an array with 10 slots so we indicate that in the square brackets. We also need to tell `new` that it is Strings in the array. Note that the size of the array is not part of the type only given to `new`.

2. Declare an array called `wins` that holds 5 integers.

CORRECT ANSWER:

```
int[] wins = new int[5];
```

EXPLANATION:

This is similar to the above except we are giving a different variable name, want an `int` array and indicate a different length to `new`.

3. Store the String `"Liverpool"` in the very first location of array called `teams` as declared above.

CORRECT ANSWER:

```
teams[0] = "Liverpool";
```

EXPLANATION:

Array positions always start counting from 0 NOT 1. So the very first cell is indexed 0. The position is indicated in the square brackets after the variable name, but other than that this is just like any other variable assignment.

4. Store the String "Real Madrid" in the last element of array called `teams` as declared above.

CORRECT ANSWER:

```
teams[9] = "Real Madrid";
```

EXPLANATION:

As numbering starts from 0, the last element of an array is always one less than the size that you gave when you declared the array variable. An array of size 10 has positions labelled from 0 to 9 NOT 1 to 10 in Java.

5. Store the value 2 in all five elements of array called `wins` (using a for loop) as declared above.

CORRECT ANSWER:

```
for (int i=0; i<=4; i++)  
{  
    wins[i] = 2;  
}
```

EXPLANATION:

To fill an array you use a loop that visits each position of the array (so the loop runs from 0 to 4 here as `wins` is of length 5.) We use the loop counter variable `i` to change the position we store the value in. The test used could also equivalently be `i<5`.

6. Prints each element of the array `teams` to the screen as declared above.

CORRECT ANSWER:

```
for (int i=0; i<=9; i++)  
{  
    System.out.println(teams[i]);  
}
```

EXPLANATION:

This time we are needing to access each element of the array `teams` which is of length 10 so we make the loop count 0 to 9 and just access the different positions using `i` as above.

7. Complete the dry run table and work out what is printed to the screen when the following code is executed.

```
int [] a = {1,1,3,1,3,1};          // L1

for(int i = 1; i<=3; i=i+1)        // L2
{
    if (a[i] == 1)                  // L3
    {
        System.out.println(i); // L4
    }
}
```

DRY RUN TABLE:

Line	a						i	i <= 3 ?	a[i] == 1 ?	print
	[0]	[1]	[2]	[3]	[4]	[5]				
L1	1	1	3	1	3	1				
L2							1	1 <= 3 ? true		
L3									1 == 1 ? true	
L4										1
L2							2	2 <= 3 ? true		
L3									3 == 1 ? false	
L2							3	3 <= 3 ? true		
L3									1 == 1 ? true	
L4										3
L2							4	4 <= 3 ? false		

CORRECT ANSWER:

1
3

EXPLANATION:

The array is initialised and the values never changed. The loop runs from 1 to 3, so the test checks in turn index positions 1, 2 and 3 of the array (note it doesn't check the 0th position or the last two positions. For each of those positions it checks if the value stored in the array at that position is 1. If it is it prints the position *i* (not the value stored there *a[i]*). So 1 is printed as a 1 is found in position 1. 2 is not printed as there is no 1 in that position and 3 is printed because there is a 1 in position 3.

Terminology

8. **Arguments** can be passed to a method – they are the pieces of data that the method needs to be given to do its job and are also known as **actual parameters**. Which three different values are passed as arguments to the method `printValue` below.

CORRECT ANSWER:

i) 5 ii) 10 iii) 7

```
public static void main (String[] param)
{
    int a = 5;
    printValue(a);
    printValue(10);
    printValue(a+2);
    return;
}

public static void printValue (int b) // Print the integer passed
{
    System.out.println("The value is " + b);
    return;
}
```

EXPLANATION:

The arguments are the actual values passed to the method. In the first call the expression given as an argument to `printValue` is the variable `a` so the argument passed is its value (5). In the second call the expression given as an argument is just the value 10. In the third call the expression given as the argument is `a+2`. This expression evaluates to 7 so the argument passed to `printValue` is 7.

9. The values passed each time the method is called need to be stored in new variables in the method that uses them. These new variables are called the **formal parameters**. What are the formal parameters of `print2values` below?

CORRECT ANSWER:

i) `x` ii) `y`

iii) Highlight the places where the formal parameters of method `print2values` are declared in the code below.

```
public static void main (String[] param)
{
```

```
    int a = 5; int c = 3;
    print2values(2, 4);
    print2values(c, a);
    print2values(a, c);
    return;
}
```

```
public static void print2values (int x, int y)
{
    System.out.println("The values are " + x + " and " + y);
    return;
}
```

10. The **return type** of a method is the type of value it passes back (or void if it is a procedure). Highlight the place where the return type of the method add3 below is declared.

```
public static int add3 (int x, int y, int z)
{
    return x + y + z;
}
```